DOT-TO-DOT MASTERCLASS: GRAPHS & ADJACENCY MATRIX

Chai aur Code DSA Series — Super Simple Hinglish Notes (Har Concept Ek Dusre Se Linked)

🔴 **DOT 1: GRAPH KYA HOTA HAI? (THE BIG PICTURE)**

Tree vs Graph — Sabse Bada Difference

> 👶 **Bachchon Jaisi Kahani:**
> • **Tree (Ped):** Jaise ek parivaar ka tree hota hai (Grandfather ➔ Father ➔ Child). Sabka ek hi Boss/Root hota hai, koi gol-gol chakkar (cycle) nahi hota.
> • **Graph (Dosti ka Network):** Jaise class ke bache aapas me haath pakad ke khade hain! Yahan koi boss nahi hai, koi bhi kisi se jud sakta hai aur gol chakkar (cycle) bhi ban sakta hai.

Graph ke do sabse important pillars hote hain:

- **Vertices (V / Nodes):** Stations ya Dost (jaise: `0, 1, 2, 3`).
- **Edges (E):** Beech ki Sadak ya Haath milana (jaise: `(0, 1)`).

Formula: `Graph G = (V, E)` (Vertices aur Edges ka joda).

🔴 **DOT 2: RASTE KE NIYAM (GRAPH KE 3 TYPES)**

Graph Ki 3 Prajatiyan (Types)

Sadak pe gaadi chalane ke alag-alag rules hote hain, waise hi Graph 3 tarah ke hote hain:

Type	Real-World Example	Rule (Niyam)
1. Undirected Graph	WhatsApp Call / Dosti	Dono taraf baatcheet (Two-way). Agar 0 ➔ 1 rasta hai, toh 1 ➔ 0 bhi jaa sakte hain.
2. Directed Graph (Digraph)	Instagram Follower / Ek-tarfa Sadak	Sirf teer (Arrow) ke direction me travel kar sakte hain. Agar 0 ➔ 1 hai, toh 1 ➔ 0 zaroori nahi.
3. Weighted Graph	Google Maps (Distance/ Toll)	Sadak pe kharcha ya doori likhi hoti hai (e.g., Delhi se Mumbai = 1400 km).

🔴 DOT 3: COMPUTER ME KAISE STORE KAREIN?

Visual Drawing → Computer Memory (Adjacency Matrix)

Computer paper pe banayi drawing nahi dekh sakta! Usse chahiye **Numbers aur Grid (Table)**.

> 💡 **Adjacency Matrix Ka Simple Funda:**
> Agar hamare paas V vertices hain, toh hum ek V x V size ka square box (2D Grid) banayenge.
> • Row = Kahan se chalna shuru kiya (**Source**)
> • Column = Kahan pahunchna hai (**Destination**)
> • Agar rasta hai ➔ Box me 1 (ya Distance/Weight) likho.
> • Agar rasta nahi hai ➔ Box me `0` likho.

Example: 4 Dost (0, 1, 2, 3)

Raste: `0-1`, `0-2`, `1-3`, `2-3` (Undirected)

From \ To	0	1	2	3
0	0	**1**	**1**	0
1	**1**	0	0	**1**
2	**1**	0	0	**1**
3	0	**1**	**1**	0

> ✨ **Golden Rule (Symmetric Property):** Undirected graph me diagonal ke dono taraf mirror image banti hai! Row 0 Col 1 par '1' hai, toh Row 1 Col 0 par bhi '1' hoga.

🔴 DOT 4: PYTHON CODE STEP-BY-STEP

Clean & Robust Object-Oriented Code

```python
class Graph:
    def __init__(self, vertices):
        self.size = vertices
        # V x V ka 2D grid banaya jisme shuru me sab 0 hai
        self.mat = [[0 for _ in range(vertices)] for _ in range(vertices)]

    def add_edge(self, src, dest, weight=1, is_directed=False):
        # Boundary check taaki invalid index error na aaye
        if 0 <= src < self.size and 0 <= dest < self.size:
            self.mat[src][dest] = weight
            if not is_directed:
                self.mat[dest][src] = weight  # Dono taraf rasta banaya
        else:
            print("Invalid Edge! Index out of range.")

    def print_matrix(self):
        print("
--- Adjacency Matrix Representation ---")
        for row in self.mat:
            # List ko string me convert karke space ke sath print kiya
            print("  ".join(map(str, row)))

# Driver Code
g = Graph(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(2, 3)

g.print_matrix()
```

🔴 DOT 5: KAB USE KAREIN AUR KAB NAHI? (INTERVIEW MINDSET)

Dense Graph vs Sparse Graph

🟢 **Faayda (Advantage):**
Check karna ki Node A aur Node B me rasta hai ya nahi, direct `mat[A][B]` dekh lo! Time Complexity = **O(1) (Instant)**.

🔴 **Nuksan (Disadvantage & Memory Waste):**
Agar 1,000 nodes hain par connections sirf 10 hain (Sparse Graph), tab bhi `1000 x 1000 = 10,00,000` dabbe banenge jisme 99.9% dabbe `0` se bhare honge! Memory Space = **$O(V^2)$**.
👉 *Solution for Sparse Graph:* Agle lecture ka topic — **Adjacency List**!